

Trabalho II — Threads em Java: Pesquisa e Aplicação

Pedro Henrique dos Reis de Melo

O que é uma Thread e qual a diferença entre processo e thread.

Uma *thread* é semelhante a um processo, mas há uma diferença fundamental entre eles: o processo é uma instância de um programa em execução, enquanto uma thread é um fluxo menor de execução dentro de um processo.

As *threads* podem ser classificadas como: *kernel-level threads* e *user-level threads*. As do tipo *kernel*, são aquelas criadas e gerenciadas pelo próprio sistema operacional. Esse tipo de *thread*, permite um controle maior e uma melhor integração com o *hardware*. Já as do tipo *user*, são criadas pela aplicação. Isso permite que uma parte do programa realize uma tarefa enquanto outra parte realiza uma outra tarefa ao mesmo tempo.

Uma maneira de entender facilmente o funcionamento de uma *thread* é observar a execução de um jogo. Nesse tipo de aplicação, diversas tarefas estão sendo executadas ao mesmo tempo. As *threads* são separadas para cada função no jogo como, desenhar imagem, transmitir áudio, etc.

As principais vantagens das *threads* são: Simplificação do modelo de programação: ao decompor uma aplicação em várias *threads* sequenciais, possibilitando realizar diversas atividades simultaneamente, podendo bloquear essas atividades de tempos em tempos. *Threads* são mais rápidas de criar e destruir: Comparado aos processos, as *threads* não possuem qualquer recurso associado a elas.

Uma *thread* possui um conjunto de comportamentos padrão, normalmente encontrados em qualquer implementação ou sistema operacional. São capazes de criar outra da mesma forma que um processo, através do método *thread-create*, onde a *thread* retorna um ID como primeiro argumento do resultado da função de criação. Uma *thread* espera a outra se sincronizar, através do método *join*. Elas podem voluntariamente desistir da *CPU*, quando não precisam de mais processamento, seja por decisão própria ou pela vontade do utilizador. Podem se replicar sem a necessidade de duplicar todo o processo, economizando memória, processamento, aproveitando o contexto de variáveis, descritores e dispositivos I/O.

Em um processo, todas as *threads* compartilham informações em comum, mas cada *thread* tem informações pertencentes somente a elas, como: espaço de endereçamento, variáveis globais, arquivos abertos, processos filhos, alarmes pendentes, sinais e tratadores de sinais, informações de contabilidade. Esses são itens que são compartilhados entre *threads*, também existem itens privados, são eles: Contador de programa, Registradores, Pilha, Estado.

Na execução de uma *thread* ela pode assumir alguns estados, como: *Unstarted*: logo após ser criada antes do *start*. *Runing*: após ser ativada ou após acionar o método *resume*. *Suspended*: após o método *Suspended()*. *Stopped*: após o método *abort*.

Como criar *Threads* no Java

Existem duas maneiras de criar uma *thread* em Java. A primeira forma é estendendo a classe *Thread* e, a segunda é implementando a *interface Runnable*.

Pelo primeiro método, é necessário criar a classe *Thread* junto com a classe que irá herdá-la. Ao instanciar a classe, a *Thread* e os objetos são criados juntos, com o objeto sendo limitado automaticamente à *thread*. Em seguida, chama-se o método *start()*, que inicia a execução da *thread* e aciona imediatamente o método *run()*.

As vantagens de criar a *Thread* por herança incluem maior simplicidade em casos básicos e a possibilidade de sobrescrever métodos da classe *Thread* se necessário.

As desvantagens são a impossibilidade de realizar “herança múltipla”. Ao estender a classe *Thread*, você perde a possibilidade de herdar outra classe. Esse método é menos flexível para o reaproveitamento de código, já que a lógica da tarefa fica acoplada à *thread*.

Esse método é mais recomendado quando há necessidade de modificar o comportamento da classe *Thread*.

No segundo método, utilizando a *interface Runnable*. Primeiramente é necessário implementar a *interface Runnable* que fornece o método *run()*. Quando o método *run()* for invocado a *thread* será iniciada.

As vantagens de utilizar a *interface Runnable* incluem maior flexibilidade, pois qualquer classe pode implementá-la. Esse método favorece a separação de responsabilidades. Isolando a lógica da tarefa da execução da *thread*.

As desvantagens são: Menor encapsulamento da execução. Já que a lógica da tarefa e o controle estão separados da classe *Thread*. E a necessidade de gerenciamento adicional, pois o controle do ciclo de vida deve ser feito externamente.

Ciclo De Vida De Uma *Thread*

O ciclo de vida representa os estados percorridos durante a execução de uma *Thread*. Estes estados são conhecidos como: *New*, *Runnable*, *Blocked*, *Waiting*, *Time Waiting* e *Terminated*.

O estado *New* é onde a *thread* nasce e é definida. Esse é o primeiro estado de uma *Thread*, e acontece antes de invocar o método *Start()*.

No estado *Runnable* o método *Start()* é invocado, deixando a *thread* pronta para ser executada e esperando seu momento de execução na fila.

O estado *Running* é o momento onde a *thread* é executada no processador. Neste momento o processador atribui um tempo de execução para a *Thread*.

No estado *Blocked*, a *thread* existe, mas está indisponível para o programador. Ou seja, está inativa.

No estado *Waiting* a *thread* para de executar e fica suspensa. Nesse estado ela não consome processamento, e fica aguardando por tempo indeterminado, até ser notificada por uma outra *thread*.

O estado *Time Waiting* é semelhante ao *Waiting*. A *thread* também fica suspensa. Porém não fica por tempo indeterminado. Quando o tempo definido expira ela volta para execução, sem a necessidade de ser notificada.

No estado *Terminated* a *thread* finalizou sua execução. Nesse estado a *thread* não pode mais ser reiniciada.

Métodos Fundamentais

Toda *thread* possui os métodos: *start*, *run*, *sleep* e *join*. Esses métodos realizam operações durante a execução.

O método *start* serve para criar uma nova *thread*. Ao chamá-lo, o método *run* é acionado internamente. Caso o método *run* for chamado diretamente o código da *thread* é executado, funcionando como um método normal.

O método *run* serve para definir o código que será executado pela *thread*, descrevendo o que acontecerá quando a *thread* for iniciada.

O método *sleep* é uma chamada estática que paralisa a *thread* por um tempo determinado. Quando esse tempo se esgota, a *thread* volta a ser executada.

O método *join* é uma forma de compor o trabalho paralelo em múltiplas etapas: criar algumas *threads*, deixá-las executar, mesclar todas e ler seus resultados.

Condições De Corrida

Uma condição de corrida (*race condition*) acontece no momento em que um comportamento de um programa depende da execução da ordem de *threads*. Quando duas *threads* acessam e modificam uma variável compartilhada simultaneamente. O resultado final pode variar dependendo de qual *thread* acessar a variável primeiro. Esse problema pode levar a resultados imprevisíveis e inconsistentes.

Um exemplo desse problema seria uma contagem simples. Onde existe a classe *Counter* com um método para incrementar um contador compartilhado.

```
public class Counter {  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```

Ao criar várias *threads* que chamam o método *increment* simultaneamente, pode ocorrer um retorno de um valor inesperado de *count*.

```
public class RaceConditionExample {  
    public static void main(String[] args) throws InterruptedException {
```

```

        Counter counter = new Counter();
        Thread t1 = new Thread(counter::increment);
        Thread t2 = new Thread(counter::increment);

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println("Contagem final: " + counter.getCount());
    }
}

```

Neste exemplo, a operação *count* não é atômica. Ela envolve leitura, incremento e escrita de volta à variável. Se duas *threads* executarem *count* simultaneamente, podem ler o mesmo valor final, resultando numa contagem incorreta.

Para resolver esse problema, podemos sincronizar o método *increment* para garantir que apenas um *thread* execute o bloco de código crítico por vez.

```

public class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}

```

Mecanismos de sincronização em Java

A sincronização é essencial em programas *multithread* para coordenar o acesso a recursos compartilhados. Existem dois principais cenários que exigem a sincronização: O acesso exclusivo a um recurso, como por exemplo, evitar que duas *threads* gravem simultaneamente em um arquivo. Outro cenário, é a cooperação entre *threads* onde uma *thread* espera por um evento desencadeado por outra.

Os métodos sincronizados são declarados com a palavra-chave *synchronized*. Durante a execução, a *thead* obtém o bloqueio do objeto. Nenhuma outra *thread* pode acessar métodos sincronizados do mesmo objeto. O objeto é desbloqueado quando a *thread* conclui o método.

O pacote *java.util.concurrent* é um conjunto de classes e interfaces que facilita o desenvolvimento de aplicações *multithread*. Ele foi criado para melhorar o desempenho e simplificar a sincronização entre *threads*, evitando o uso excessivo de *synchronized* e oferecendo estruturas mais modernas e flexíveis.

O *AtomicInteger* é uma classe que representa um inteiro que pode ser atualizado de forma atômica. Ela usa instruções de baixo nível da CPU para garantir que operações como incremento ou decremento sejam seguras mesmo com várias threads acessando ao mesmo tempo.

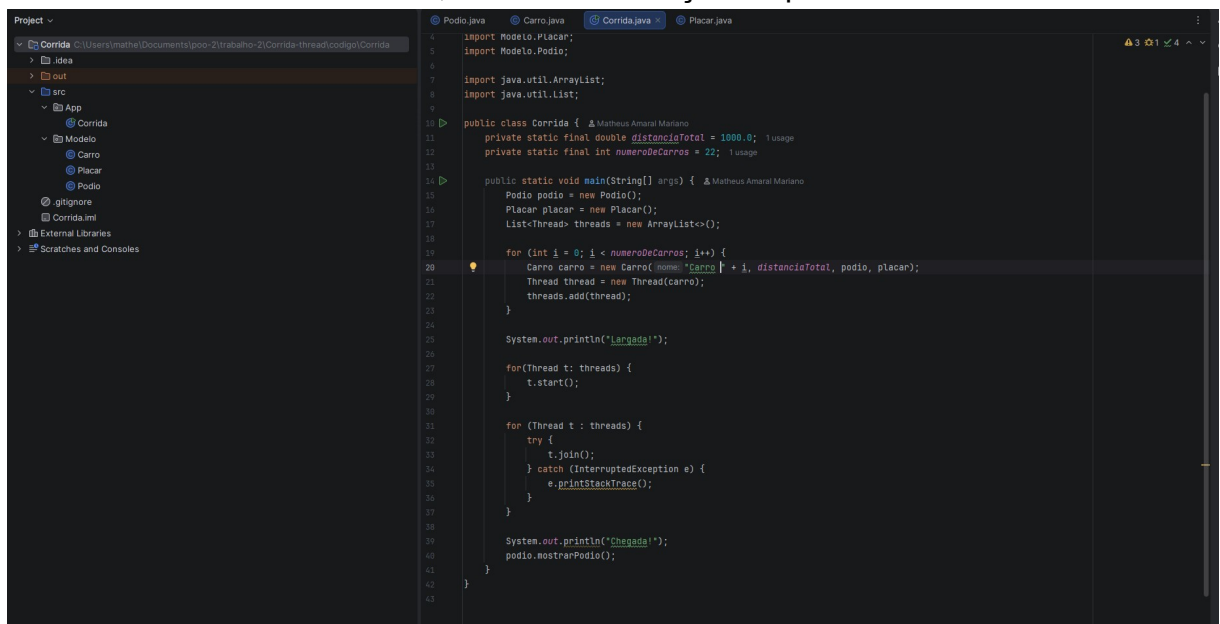
O *Reentrantlock* é uma implementação mais avançada de bloqueio (lock) que substitui ou complementa o *synchronized*. Ele é útil quando é necessário mais controle sobre o bloqueio, tempo limite ou prioridade de acesso.

principais dificuldades e aprendizados durante a pesquisa

Compreender o funcionamento das *threads* e sua implementação em Java foi um grande desafio. Durante a pesquisa, aprendi como as *threads* funcionam no sistema operacional, seus métodos fundamentais e ciclo de vida que percorrem. Também aprendi a criar threads em Java e a lidar com problemas comuns, como as condições de corrida, além de entender a importância da sincronização para garantir a execução correta e segura de programas concorrentes.

Desenvolvimento: Simulador de Corrida de Carros

O projeto é composto por quatro classes: Corrida, Carro, Placar e Podio. A classe Corrida é a principal e responsável pela lógica da competição. Ela define a distância total da prova, o número de carros participantes, cria as *threads* para cada carro e executa outros métodos, como o de exibição do pódio.



```
1  Podio.java  Carro.java  Corrida.java  Placar.java
2  import Modelo.Placar;
3  import Modelo.Podio;
4
5  import java.util.ArrayList;
6  import java.util.List;
7
8
9
10 public class Corrida { @Matheus Amarel Mariano
11     private static final double distanciaTotal = 1000.0; //usage
12     private static final int numeroDeCarros = 22; //usage
13
14     public static void main(String[] args) { @Matheus Amarel Mariano
15         Podio podio = new Podio();
16         Placar placar = new Placar();
17         List<Thread> threads = new ArrayList<>();
18
19         for (int i = 0; i < numeroDeCarros; i++) {
20             Carro carro = new Carro(nome: "Carro" + i, distanciaTotal, podio, placar);
21             Thread thread = new Thread(carro);
22             threads.add(thread);
23         }
24
25         System.out.println("Largada!");
26
27         for (Thread t : threads) {
28             t.start();
29         }
30
31         for (Thread t : threads) {
32             try {
33                 t.join();
34             } catch (InterruptedException e) {
35                 e.printStackTrace();
36             }
37         }
38
39         System.out.println("Chegada!");
40         podio.mostrarPodio();
41     }
42 }
```

A classe Carro é responsável pela lógica aplicada à cada veículo, definindo seu nome, posição e classificação. Ela retorna tudo o que acontece com o carro durante a corrida. Se está no pit stop, quantos metros percorreu e se cruzou a linha de chegada.

O método utilizado para a criação da *thread*, foi a *interface Runnable*. O principal motivo da escolha desse método foi sua facilidade de implementação, já que basta implementar a interface e utilizar o método *run()*.

```
1 package Modelo;
2
3 public class Carro implements Runnable{ 3 usages Matheus Amaral Mariano
4     private String nome; 6 usages
5     private Double distanciaTotalCorrida; 6 usages
6     private Double distanciaPercorrida; 8 usages
7     private Podio podio; 2 usages
8     private Placar placar; 2 usages
9
10    public Carro (String nome, Double distanciaTotalCorrida, Podio podio, Placar placar) { 1 usage Matheus Amaral Mariano
11        this.nome = nome;
12        this.distanciaTotalCorrida = distanciaTotalCorrida;
13        this.distanciaPercorrida = 0.0;
14        this.podio = podio;
15        this.placar = placar;
16    }
17
18    @Override Matheus Amaral Mariano
19    public void run() {
20        while (distanciaPercorrida < distanciaTotalCorrida) {
21            double incremento = 10 + Math.random() * 40;
22            distanciaPercorrida += incremento;
23
24            if(distanciaPercorrida > distanciaTotalCorrida) {
25                distanciaPercorrida = distanciaTotalCorrida;
26            }
27
28            System.out.println(nome + " andou " + String.format("%.2f", incremento) +
29                " metros e já percorreu " + String.format("%.2f", distanciaPercorrida) +
30                " de " + distanciaTotalCorrida + "metros"
31        );
32    }
```

A classe Placar é responsável por exibir a classificação dos carros durante a corrida, mostrando sua colocação exata como: 1°, 2°, 3°, etc.

```
1 package Modelo;
2
3 import java.util.HashMap;
4 import java.util.List;
5 import java.util.Map;
6
7 public class Placar { 5 usages Matheus Amaral Mariano
8     private Map<String, Double> posicoes = new HashMap<>(); 2 usages
9
10    public synchronized void atualizar(String nomeCarro, double distancia) { 1 usage Matheus Amaral Mariano
11        posicoes.put(nomeCarro, distancia);
12        mostrarPlacar();
13    }
14
15    public void mostrarPlacar() { 1 usage Matheus Amaral Mariano
16        System.out.println("\n == PLACAR ==");
17        List<Map.Entry<String, Double>> ordenados = posicoes.entrySet().stream()
18            .sorted(((Entry<String, Double> c1, Entry<String, Double> c2) -> Double.compare(c2.getValue(), c1.getValue())))
19            .toList();
20
21        for (int i = 0; i < ordenados.size(); i++) {
22            Map.Entry<String, Double> entry = ordenados.get(i);
23            int posicao = i + 1;
24            System.out.println(posicao + "° lugar: " + entry.getKey() +
25                " -> " + String.format("%.2f", entry.getValue()) + " metros");
26        }
27
28        System.out.println("=====\n");
29    }
30 }
```

A classe Podio é responsável por exibir os vencedores da corrida. Organizados na ordem de primeiro, segundo e terceiro.

```

1 package Modelo;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class Podio { 5 usages  ⚙ Matheus Amaral Mariano
7     private List<String> vencedores = new ArrayList<>(); 5 usages
8
9     public synchronized void registrarVencedor(String nomeCarro) { 1 usage  ⚙ Matheus Amaral Mariano
10         if (vencedores.size() < 3) {
11             vencedores.add(nomeCarro);
12             System.out.println(nomeCarro + "ficou em " + vencedores.size() + " º lugar!");
13         }
14     }
15
16     public void mostrarPodio() { 1 usage  ⚙ Matheus Amaral Mariano
17         System.out.println("\n == PÓDIO ==");
18         for (int i = 0; i < vencedores.size(); i++) {
19             System.out.println((i + 1) + "º lugar: " + vencedores.get(i));
20         }
21     }
22 }

```

Link do repositório no gitHub

<https://github.com/Ma27Am12/Corrida-thread.git>

Referências

BARANCELLI, Gaspar. *Entendendo Race Conditions em Java: Conceitos e Soluções.* Gaspar Barancelli, 2021. Disponível em: <<https://gasparbarancelli.com/post/entendendo-race-conditions-em-java-conceitos-esolucoes>>. Acesso em: 21 set. 2026.

CONCEITOS TECH. *Threads em Java: O que são e como criar?* Disponível em: <<https://conceitos.tech/tutoriais/java/concorrenca-threads-e-desempenho/threads-java-como-criar/>>. Acesso em: 21 set. 2026.

FUNDAMENTOS JAVA. *Sincronização e usando métodos sincronizados.* DEV Community, 2021. Disponível em: <<https://dev.to/fundamentosjava/sincronizacao-e-usando-metodos-sincronizados-1p01>>. Acesso em: 21 set. 2026.

REIS, Alex. *O básico de Threads no Java.* DEV Community, 2020. Disponível em: <<https://dev.to/alexreis/o-basico-de-threads-no-java-1881>>. Acesso em: 21 set. 2026.

WIKIPÉDIA. *Thread (computação).* Disponível em: <[https://pt.wikipedia.org/wiki/Thread_\(computa%C3%A7%C3%A3o\)](https://pt.wikipedia.org/wiki/Thread_(computa%C3%A7%C3%A3o))>. Acesso em: 21 set. 2026.